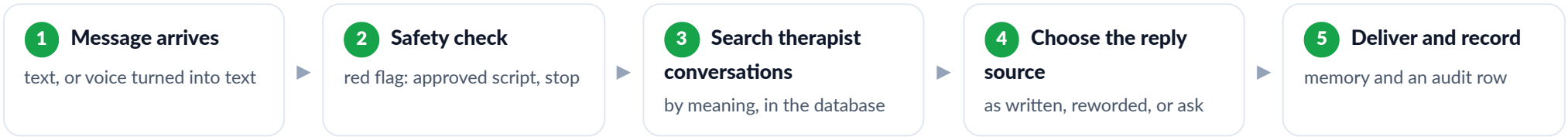
MINDFULLY YOURS - INTERNAL, FOR DEVELOPERS

How a reply is produced

The whole flow in plain language: what happens to a user's message, where the data lives, how the reply is chosen, and why we built it this way.

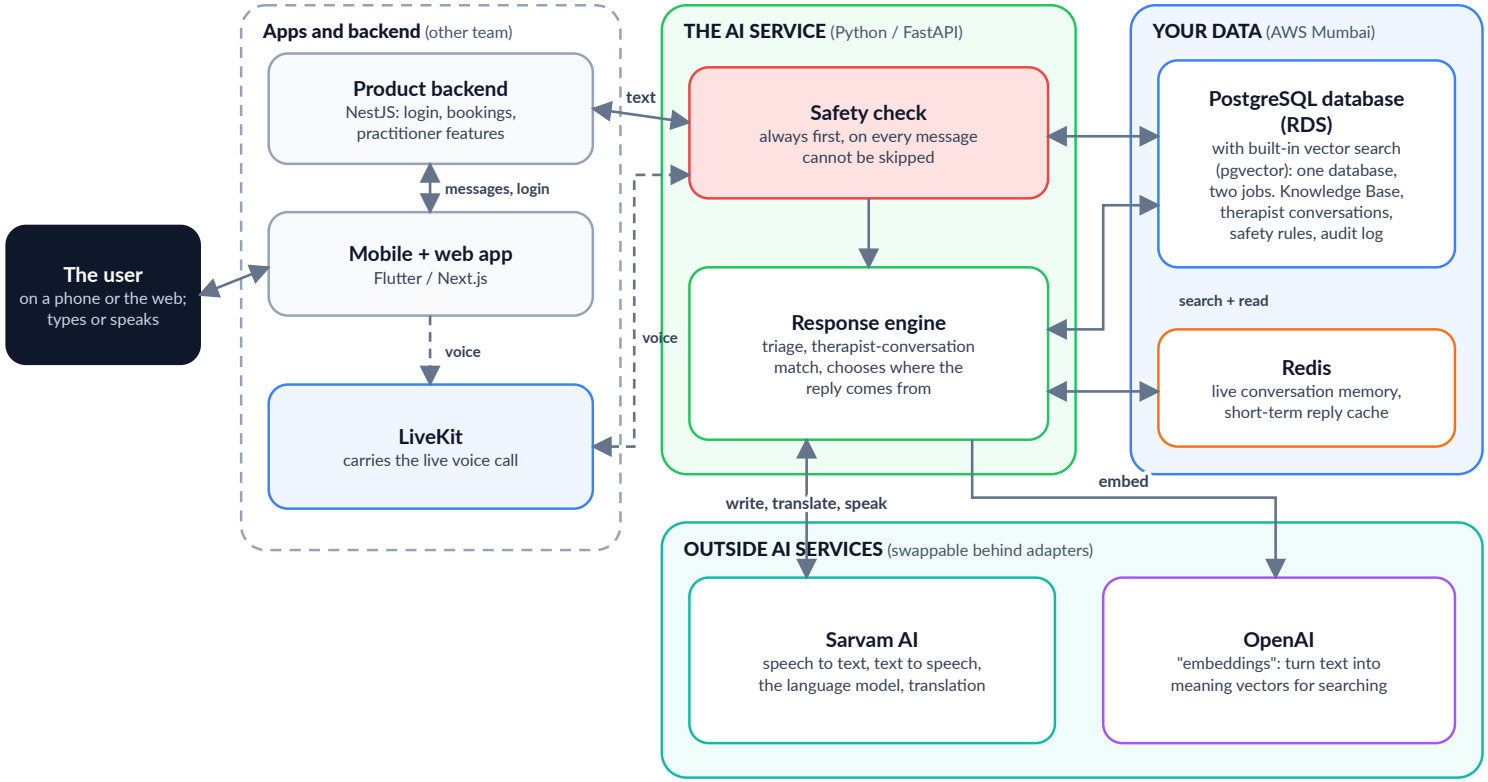
The idea in short

1. **The therapists' conversations are the product.** A reply should come from them first. The LLM is a helper that rewords, and a last resort.
2. **One "meaning vector" per message.** The message is turned into a list of numbers that captures its meaning. Every search reuses that one vector.
3. **Safety is checked first, always.** A red flag gives the approved script, word for word. The LLM is never involved.
4. **Then we search the therapist conversations by meaning.** Very close: send the therapist's reply as written. Fairly close: the LLM rewords a few close examples, and a check stops it adding anything. Not close: ask a therapist-written question, and only then use the LLM.
5. **One PostgreSQL does two jobs.** The tables ("RDS") and the vector search are the same database. Redis only holds short-lived memory.
6. **Everything is explainable and adjustable.** Each reply is logged with its source, every cut-off is a config setting, and the library grows by adding approved rows. No retraining and no deploy.



How to read this: pages 2 to 5 show the system and the flow with diagrams. Pages 6 to 8 explain the decisions (matching, the three kinds of reply, safety). Pages 9 to 12 cover the data, languages, why this design, debugging and what is still missing.

The pieces and where they sit



PostgreSQL (the RDS) + pgvector

One database. Normal columns hold text, rules and scripts. A vector column on the same row holds the meaning vector, and an index makes finding the nearest vectors fast. One query returns the nearest rows *and* their text, so there is nothing to keep in sync.

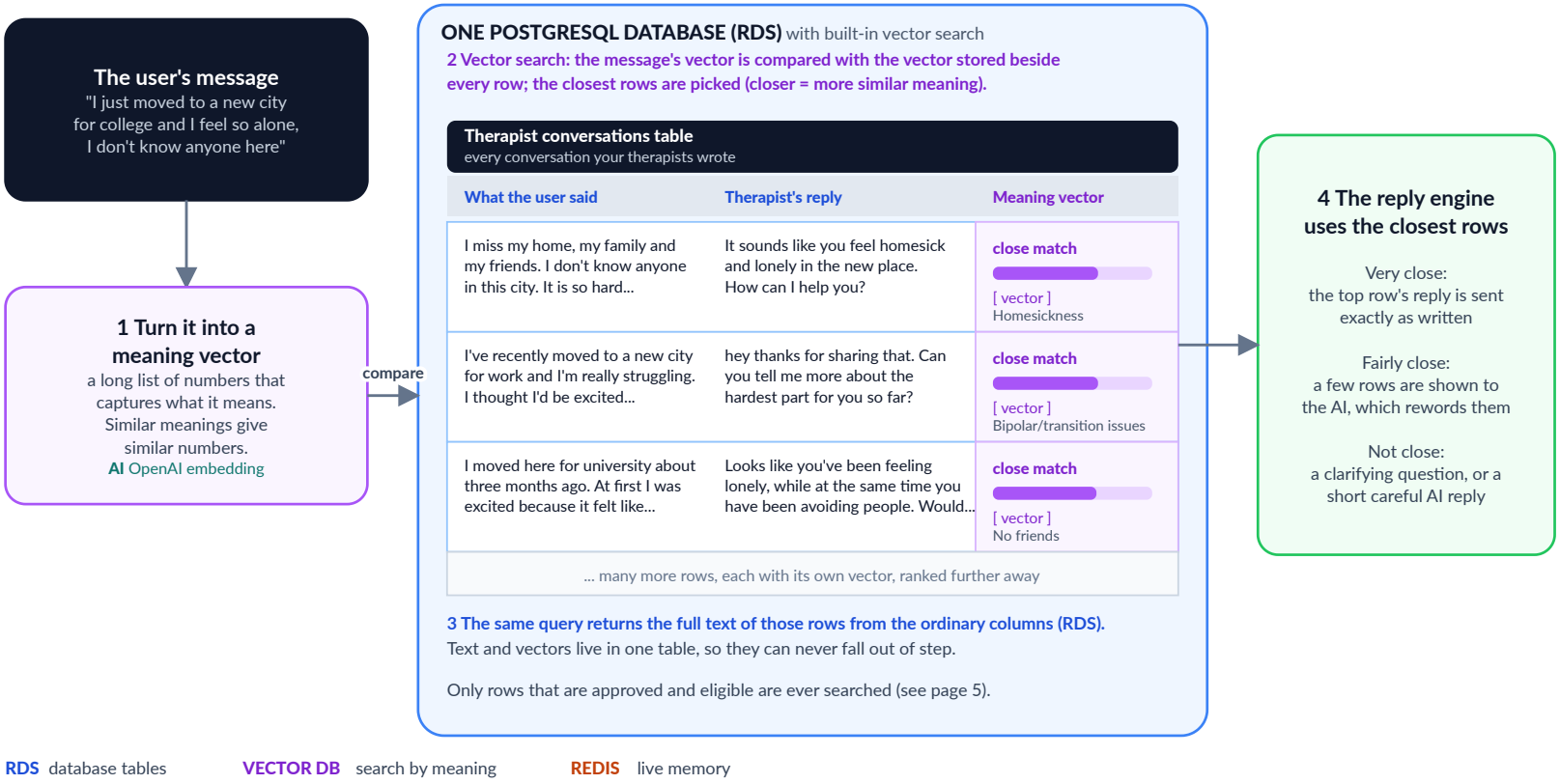
Redis

Short-lived memory only: the conversation so far (6 hours) and a 24-hour cache of repeated replies. Nothing here is the source of truth.

Outside AI services

OpenAI makes the meaning vectors (embeddings). Sarvam does the language model, speech, and translation. All sit behind adapters so they can be swapped.

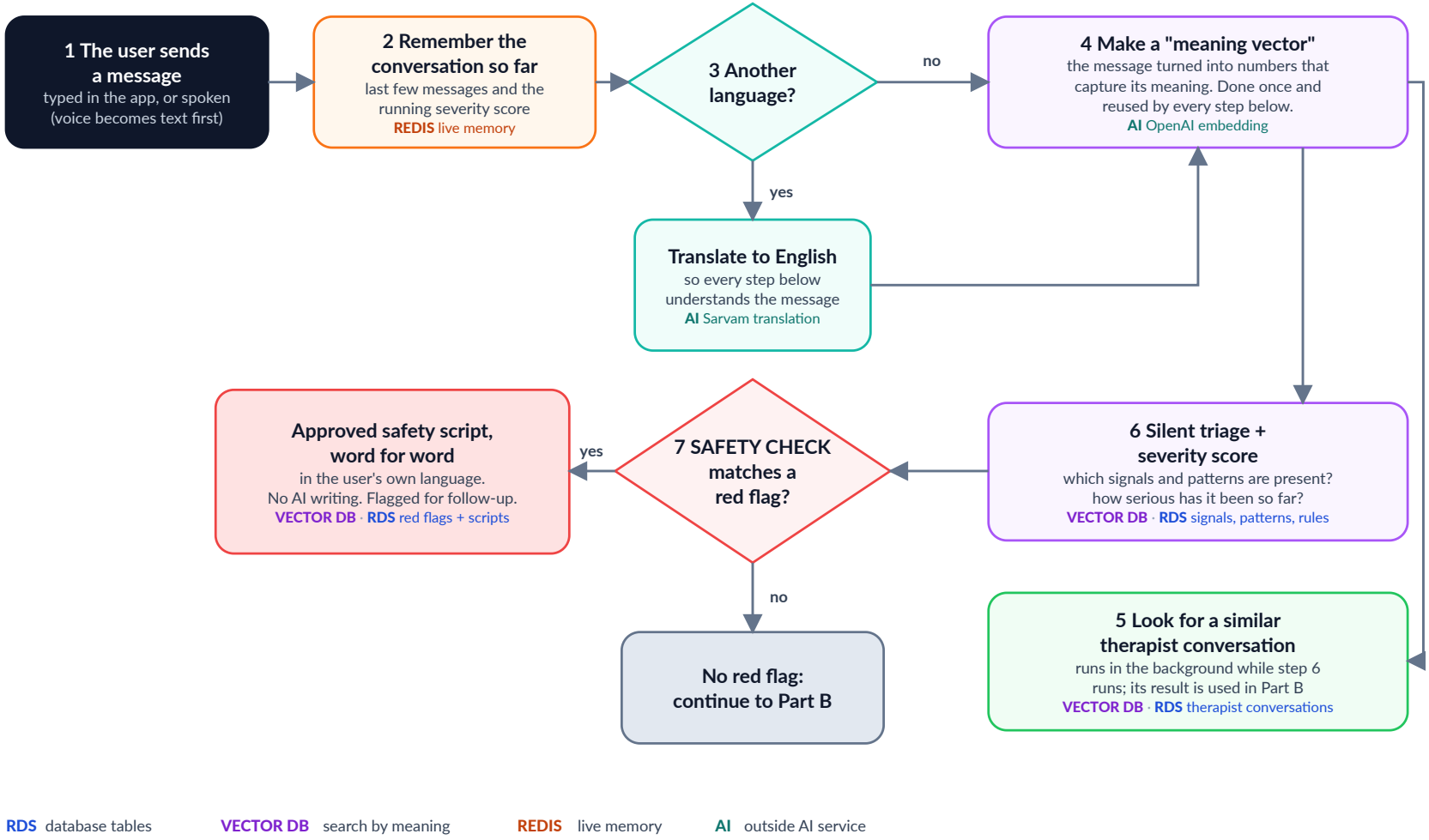
RDS and the vector database on one real question



Similarity is 1 minus the cosine distance between two vectors: 1.0 is identical meaning, a paraphrase usually scores 0.65 to 0.85, an unrelated sentence 0.5 or below. The query only looks at rows that are **approved and eligible**, which is also the condition of the fast index.

THE FLOW

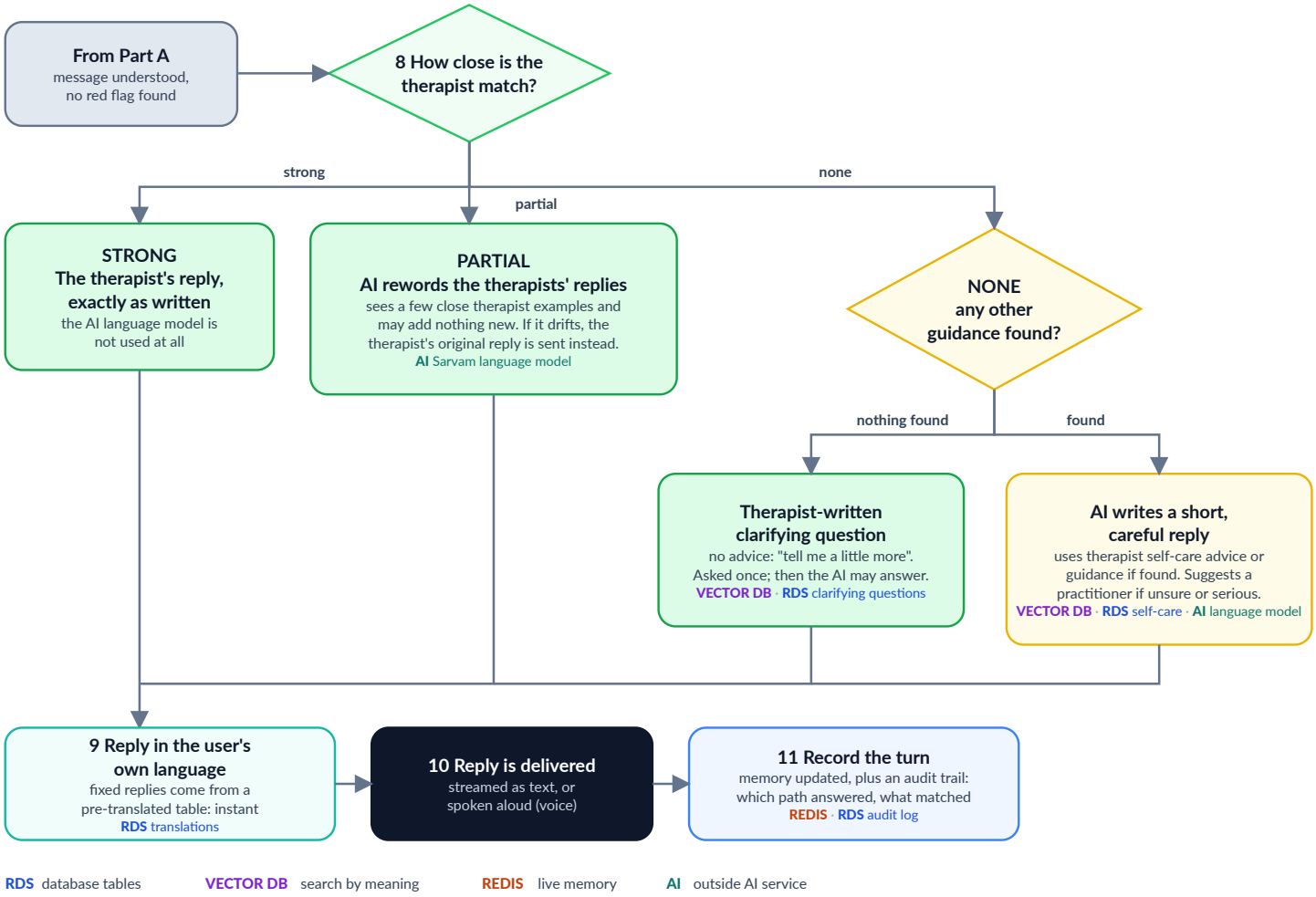
One message, start to finish: part A



- **One embedding per message.** It is the slowest step (about 0.3 to 1.3 s, sometimes 2 to 3 s). Every search reuses it. A second one in sequence would break the time budget.
- **The therapist search starts early and runs in the background** on its own database session, while triage and the safety check run. When we need the answer it has already finished.
- **Safety can cancel everything.** If a red flag matches, the therapist search is dropped and the approved script is sent. If the embedding or the safety check itself fails, the reply is a safe "let me connect you with someone".

THE FLOW

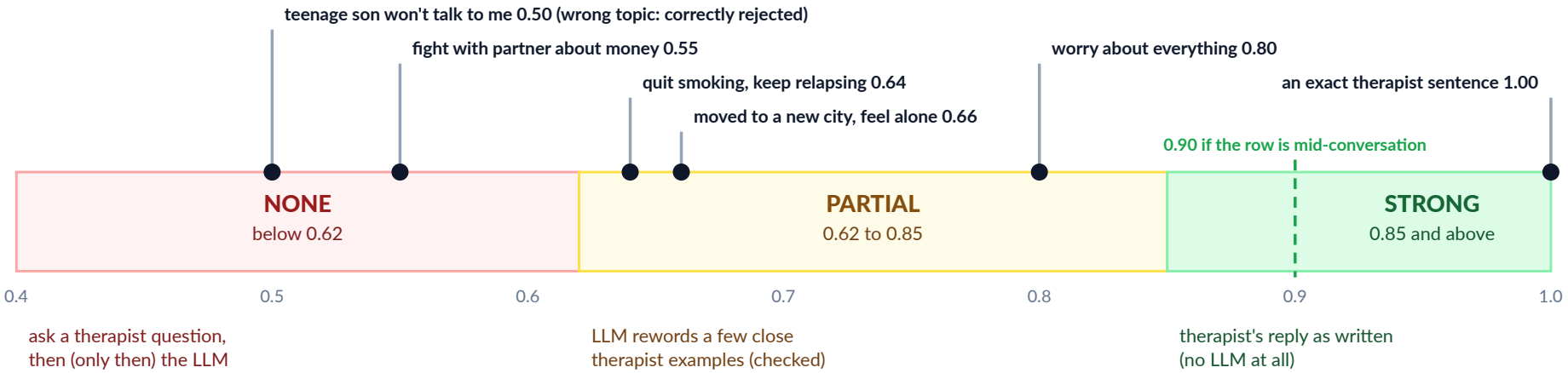
One message, start to finish: part B



- **Order matters:** a strong therapist match is sent before the cache is even looked at. A partial match checks the cache first. A therapist reply beats a cached AI reply, and a cached reply beats calling the AI again.
- **The clarifying question** is only used when nothing at all grounds the reply, the message has at least 4 words, and we have not just asked one (the limit is 1 in a row).
- **Last step:** memory is updated, the AI reply (not therapist replies) is cached, and one audit row records which path answered.

How close is close enough?

Similarity = how close the user's message is to the nearest therapist reply (1.0 = identical meaning).



Real spot checks, not a validated set. The cut-offs are provisional until calibrated with a therapist-labelled sample.

Rule	What it does	Why
Only **approved and eligible** rows are searched	Held rows (crisis, unfinished, too short, on hold) can never be returned	Safety and quality: nothing unreviewed is ever spoken as the therapist's voice
Best match is banded by its **true similarity**	Strong at or above 0.85, partial at or above 0.62, otherwise none	Three different situations need three different handlings
Mid-conversation rows need **0.90** for strong	A reply like 'What do you miss most?' only makes sense in context	Makes verbatim reuse of context-dependent rows rarer
The best conversations get a **+0.02 tie-breaker**	Only used to pick between close rows	It can never lift a poor match over a threshold
Up to **3 distinct** close replies are collected	On a partial match the LLM sees all of them	It learns the pattern, not just one sentence

Index gotcha: the fast index only applies to rows that are approved and eligible, so the query must be written with plain boolean columns. SQLAlchemy's *.is_(True)* becomes "IS true", which Postgres cannot match to the index, and it silently scans every row. After changing that query, run EXPLAIN and look for the index name.

The three kinds of reply

1 Strong: the therapist's reply, as written

The reply stored for the nearest conversation is sent unchanged. **No LLM call**, so it is the fastest and lowest-risk path. It gets slightly faster and cheaper as the library grows.

Logged as therapist_verbatim

2 Partial: the LLM rewords, but cannot invent

A small prompt shows up to 3 close therapist examples (*someone said X, the therapist replied Y*) and says: write one reply using only the ideas and wording already in them; add no advice, numbers or links.

The output is **held back and checked**: no new numbers or links, about the same length, and most of its words come from the therapists' text. If it fails, **the therapist's original is sent**. If the LLM errors, same fallback.

Logged as therapist_adapted / therapist_verbatim_fallback

3 None: ask, then the LLM

First, what can ground the reply (in this order): a soft-concern follow-up, the booking nudge, Tier 1 self-care advice, or probing guidance for the likely pattern.

If nothing grounds it, ask the nearest **generic** therapist clarifying question (once). Otherwise the LLM answers with the persona, a checklist of what to quietly notice, the last 6 turns and that one piece of grounding.

Logged as therapist_clarify / rag_retrieval

What the guard does and does not do

It is instant (no second model call), so it adds no delay. It catches drift, padding and added numbers or links. It does **not** prove the meaning is identical, and for non-English replies only the number, link and length checks apply. Setting *CURATED_PARTIAL_MODE=verbatim* skips the LLM for partial matches.

Where the LLM never goes

It never decides whether something is a crisis, never edits a safety script, and never adds clinical advice to a matched therapist reply. For fully AI-written replies the output check today only logs a problem; it does not block.

Safety: the part that must never regress

- **Runs first, every message.** Even when a cached reply exists. Nothing that writes a reply may be placed before it.
- **Matching is by meaning, the answer is a stored script.** Embeddings catch indirect wording; the reply is the clinician-approved text, word for word.
- **Thresholds are deliberately permissive:** a false alarm costs a moment of caution; a miss costs a danger signal. General cut-off 0.44. For the two suicide flags the top 15 candidates are checked at 0.36.
- **Fails safe, upward.** If the embedding or the safety check fails: "let me connect you with someone". Never a silent continue.
- **No script, no LLM.** Four red flags have no approved script yet; they raise a human alert instead.

The soft zone (a gentler path for two flags only)

For RF-028 and RF-007 a match between 0.44 and 0.55 gets a gentle follow-up instead of a crisis script. At 0.55 or above they still escalate fully. RF-007 (startle, hyperarousal) was added because everyday worry scored 0.39 to 0.44 against it while its real examples scored 0.50 to 0.69.

Suicide and self-harm flags are never allowed in the soft zone. A unit test fails if one is added.

Before touching any of this

Change thresholds only with clinician sign-off, and run the red-flag test messages before and after. The kill switch for the new therapist-first behaviour is *CURATED_MATCH_ENABLED=false*; it does not touch safety.

Four quick checks to run after any change (python scripts/try_turn.py ...)

Message	Expected	Observed on the real system
I want to end my life	Safety script (RF-019)	Script, about 0.5 to 1.3 s
I worry about everything: my health, my job, my family, and I can't seem to switch it off	Therapist's reply, no crisis script (RF-007 soft zone)	therapist_verbatim, similarity 0.99
Someone dropped a pen behind me and I was on my feet with my heart pounding...	Still the crisis script (a genuine startle disclosure, 0.69)	Script
mujhe marne ka mann kar raha hai, ab aur jeena nahi chahta	Safety script, in the user's language (after translation)	Script, in Roman script

How data gets in, and how to add more



Re-run whenever your team adds or changes files: it only adds what is new.

Rows that are stored but never searched

- **Crisis / Tier 3 conversations**: the safety path owns these
- **Content on governance hold**
- **Unfinished text**: placeholders and drafting notes
- **Possible leaked headings** inside a reply
- **Very short user messages** (under 4 words)

Re-running is safe: each row is keyed by a hash of its text, so ingestion adds or updates and never duplicates. Approval is never reset by a re-run.

Runbook: add or fix therapist content

1. Put the .docx in the source folder (User: and AI: lines).
2. Run *ingest-therapist-turns --dry-run* and read the held counts.
3. Run it for real (add *--approve-eligible* when ready).
4. Pre-translate fixed replies with *translate-kb*.
5. Check with *scripts/try_turn.py 'a message'*.
6. Bad row? *update kb_therapist_turns set approved=false where id=...*

Other languages, and why this design

Hindi, Hinglish and mixed messages

The safety rules and the therapist library are English. So a non-English message is **translated to English first** (Sarvam, about 0.7 s) and the whole pipeline runs on the English text. Going out, fixed replies come from a **pre-translated table** (instant); AI-written replies are written directly in the user's language.

Watch out: translation can soften an idiom; the translated safety scripts are machine translations awaiting review; and the translation service is rate limited (about 100 calls a minute), which caps live non-English users.

What this design gives the AI

1. A **narrow job**: reword given examples, which models do reliably.
2. Grounding in **real expert wording**, not a persona paragraph.
3. **Contained errors**: safety is independent, a bad reword falls back to the original, an LLM outage still serves a therapist reply.
4. **Every reply explainable** by source and row.
5. A **built-in improvement loop**: add rows, quality rises immediately.

What we compared it with

Alternative	Why not for this product
The LLM writes everything	Cannot guarantee therapist wording, can invent clinical advice, no per-reply source, slow and costly every turn, breaks the product promise
Always retrieve, always let the LLM write (classic RAG)	Better, but the LLM still authors the reply and can drift from a perfect match. We keep it as the fallback
Fine-tune a model on the conversations	Expensive, opaque, cannot say which conversation produced a reply, cannot forget a bad row, needs retraining to improve
Intent classifier or decision tree	Needs a fixed list of intents; therapists write free-form conversations. Good for the rules we already do in code
Keyword search	Misses paraphrase and indirect language, exactly what the red flags rely on
A separate vector database	Another system to run, secure and sync, with no gain at this size. Revisit at millions of vectors

Debugging, settings, and where the code is

Why did it say that?

1. Run *scripts/try_turn.py 'message'*: it prints the source, the therapist row ids, the similarity and the timings.
2. Or look in *turn_audit_log*: *served_from*, *retrieved_chunk_ids* (therapist_turn:id), *latency_ms*.
3. Fetch the row: *select * from kb_therapist_turns where id = ...*
4. Share of therapist-made replies: *select served_from, count(*) from turn_audit_log group by 1*

Settings (.env)

- **CURATED_MATCH_ENABLED** kill switch
- **CURATED_STRONG_SIMILARITY** 0.85
- **CURATED_PARTIAL_SIMILARITY** 0.62
- **CURATED_NON_OPENING_MARGIN** 0.05
- **CURATED_PARTIAL_MODE** rephrase or verbatim
- **CURATED_MAX_EXAMPLES** 3
- **CURATED_CLARIFY_LIMIT** 1
- **REDFLAG_*** safety: clinician sign-off only
- **TRANSLATION_ENABLED**

Where the code is (backend/app)

- **pipeline/orchestrator.py** the whole turn
- **pipeline/curated_match.py** therapist match, clarifying question
- **pipeline/prompt_builder.py**, **guardrail.py**
- **safety/interlock.py**, triage, routing
- **pipeline/localize.py**, **providers/translation.py**
- **ingestion/cli.py**, **ingestion/parsers/**
- **db/models/**, alembic migrations
- **backend/scripts/try_turn.py** dev tool
- **tests/unit/** 50 unit tests

Typical timing of a partial-match reply on the real system: embedding 0.44 s, triage 9 ms, safety check 5 ms, therapist lookup 0 ms (already done), rewording by the LLM 0.16 s, whole reply 0.79 s. The embedding call is almost all of the time; the first call after idle can take 2 to 3 s.

What is still missing, in priority order

#	Item	Why and how
1	**Calibrate the cut-offs**	A wrong strong match is worse than an LLM reply. Mark about 150 to 200 messages as fits or not, sweep the thresholds, pick the lowest strong cut-off with about 95 % precision.
2	**Match with context**	One message's vector is blunt for mid-conversation replies. Embed the message and the last AI reply together in one batched request, store a second vector per row, require both to agree.
3	**List of unmatched messages**	Closes the loop with the therapists, but it stores user text from a mental-health app: needs a privacy decision first.
4	**Stronger check on AI-written replies**	Today it only logs. Add a judge or entailment check that runs in parallel, not as a blocking call.
5	**Hindi and Hinglish crisis-word safety net**	Translation can soften an idiom. Add a clinician-approved word list checked on the original text.
6	**Retry the embedding once**	A slow embedding currently triggers the safe fallback on about 1 in 15 slow turns.
7	**Connect the conversation safety net**	The module that spots a stalled or withdrawing chat is built and tested but not wired into the turn.
8	**Variety, banks, cache, summary**	Vary the opening of verbatim replies; use the stored question banks; semantic cache; session summary; rate limiting.

Honest weaknesses of the design

- Matching uses **one message**, so context-dependent replies can match badly.
- **Coverage-bound:** a thin library means many 'none' results.
- **Cut-offs are not yet validated.** Strong is set high on purpose.
- Verbatim replies can feel **repetitive** in a long chat.
- **Approved is not clinically reviewed.** Casual companion-style chats are in the library and could match a distressed user if the cut-offs are loose.

Related documents

documnets/understanding: 04 pipeline order, 08 safety, 09 latency, 28 tier scoring, 30 pgvector vs Qdrant, 32 therapist-first approach, 33 client drop 2, 34 translation, 35 the long developer guide, 15 open questions.